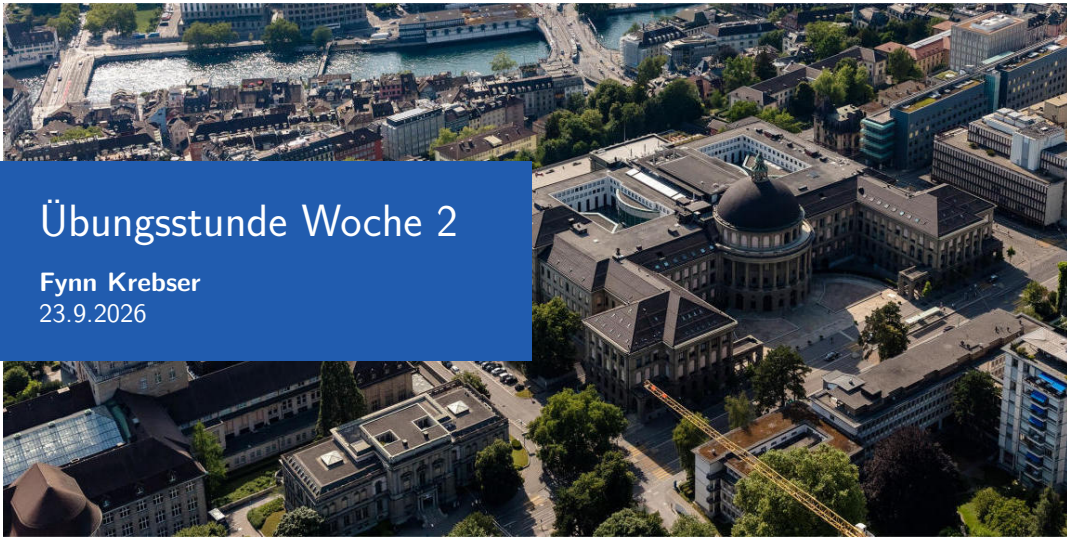


Übungsstunde Woche 2

Fynn Krebser

23.9.2026



Warmup Quiz

1. Was ist der Unterschied von `x++` und `++x`?
2. Was gibt `-2 + 1u` zurück? (`u` steht für `unsigned int`)
3. Wie lässt sich nicht (einfach und logisch) anders schreiben?

Fragen letzter Woche

- Wieso muss man `iostream` überhaupt inkludieren?
⇒ es gibt mehrere Möglichkeiten, wie man Ein- und Ausgabe in C++ machen kann. `iostream` ist eine davon.
- Wieso benötigt man ein `return 0;` am Ende der `main()` Funktion?
⇒ `main()` ist eine Funktion (siehe Später). Technically ist es erlaubt diese hier wegzulassen, jedoch ist es einheitlicher, wenn man sie immer schreibt.

Congratulations!



Feedback zur Serie muss Leider bis nächste Woche warten, da ihr noch Zeit hättet...

Ziel von Heute

- Wie werden Ausdrücke in C++ ausgewertet?

Division

- Division ist nicht gleich Division
- Integer Division: $5 / 2 = 2$
- Floating Point Division: $5.0 / 2.0 = 2.5$

Klassische Fangfrage an der Prüfung

Modulo Operator

- Gibt den Rest einer Division zurück
- Beispiel: $5 \% 2 = 1$

Funktioniert nur mit Integern. Floating Point Zahlen können nicht modulo genommen werden.

Rekonstruktionen von Zahlen

Gesucht ist eine Zahl x sodass:

$$x/4 = 3 \quad \text{und} \quad x\%4 = 2.$$

$$x = 4 \cdot 3 + 2 = 14.$$

Wie können wir das in C++ umsetzen?

Was macht der folgende Code?

```
1  int a;  
2  std::cin >> a;  
3  if (a % 2 == 0) {  
4      std::cout << "Yes" << std::endl;  
5  } else {  
6      std::cout << "No" << std::endl;  
7  }
```

Falls ihr das herausgefunden habt, begründet wieso.

Last Three Digits

<https://expert.ethz.ch> 

Probiert es zuerst mit Stift und Papier an den Zahlen 789 und 12345 aus. Versucht zu verstehen wie ihr das mit einem Algorithmus lösen könnt.

Logische Ausdrücke

- Neuer Datentyp: `bool`
- Kann nur die Werte `true` oder `false` annehmen
- Ziel: Ausdrücke wie `(x > 5) && (y < 10)` korrekt auswerten

Auswertungsreihenfolge

Operator	Präzedenz
*, /, %	5
+, -	6
<, <=, >, >=	9
&&	14
	15



**Bindet
stärker**

Wenn ihr unsicher seid, setzt Klammern.

Auswertungsreihenfolge

Operator	Präzedenz
*, /, %	5
+, -	6
<, <=, >, >=	9
&&	14
	15

1. $3 < 4 + 1 \ \&\& \ 2 < 3$

2. `false && false && true`

3. `false || true && true`

4. $7 * 3 + 1 < 5 \ || \ 2 < 3$

Short-Circuit Evaluation

- Was gibt $0 \cdot (1 - 981^{491} + 38.5)$?
- Von Anfang weg ist klar, dass das Ergebnis 0 ist.
- In C++ wird der zweite Teil des Ausdrucks nicht ausgewertet, wenn das Ergebnis des ersten Teils bereits das Ergebnis bestimmt. **Short-Circuit Evaluation**
- Erste Option: `if (x > 5 && y < 10)`
- Zweite Option: `if (x > 5 || y < 10)`

Short-Circuit

Was passiert hier, Schritt für Schritt, wenn $x = 1$?

1. $x == 1 \ || \ 1 / (x - 1) < 1$ für $\text{int } x = 1$
2. $!(1 < 2) \ \&\& \ x == 1$ für $\text{int } x = 1$
3. $(x == 1) \ || \ (1 / (x - 1) < 1)$ für $\text{int } x = 1$

Bool-Reductions

Minimiert die folgenden Ausdrücke, wobei a und b bools sind, und n ein int.

1. `b == true && !(n<0)&& (n != 0)&& (n != 6)&& (n < 6)`
2. `a && !(a == true)|| (b == false)`
3. `!(a != false && !(b == false))== false)`

Gültige Ausdrücke

- Ein Ausdruck ist *gültig*, wenn er syntaktisch korrekt ist und die Datentypen der Operanden zusammenpassen.
- Beispiele:
 - $5 + 3$ ist gültig, da beide Operanden int sind.
 - $5.0 + 3$ ist gültig, da der int Operand in einen float konvertiert wird.
 - $5 / 0$ ist gültig, aber führt zu einem Laufzeitfehler (Division durch Null).
 - $5+$ ist nicht gültig, da der Ausdruck unvollständig ist.

Gültige Ausdrücke

Welche der folgenden Ausdrücke sind gültig, welche nicht? (x und y sind int Variablen)

1. $1*(2*3)$
2. $(x=1)$
3. $(1$
4. $(a*3) = (b*5)$

L-Values und R-Values

- Ein Ausdruck ist ein *L-Value*, wenn er eine Speicheradresse hat, d.h. man kann ihn auf der linken Seite einer Zuweisung verwenden.
- Ein Ausdruck ist ein *R-Value*, wenn er keinen Speicherort hat, d.h. man kann ihn **nicht** auf der linken Seite einer Zuweisung verwenden.

L und R-Values

Welche der folgenden Ausdrücke sind L, welche R-Values? (x und y sind int Variablen)

1. $1*(2*3)$
2. $(x=1)$

Gemischte Ausdrücke

`bool < int < unsigned int < float < double`

→
Allgemeinerer Datentyp

Grundregel: Wenn ein Ausdruck mehrere Datentypen enthält, wird der allgemeinere Datentyp verwendet.

Achtung: `unsigned int` ist allgemeiner als `int`!

Beispiel: $5.0/2 = 2.5$ und $5/2.0 = 2.5$, aber $5/2 = 2$.

Gemischte Ausdrücke

Welche der folgenden Ausdrücke sind gültig, welche nicht? (x und y sind int Variablen)

1. `(y++ < 0 && y < 0) + 2.0`
2. `y = (x++ = 3)`
3. `3.0 + 3 - 4 + 5`
4. `5 % 4 * 3.0 + true * x++`

Gemischte Ausdrücke

Welche der folgenden Ausdrücke sind L-Values, welche R-Values? (x und y sind int Variablen)

1. `(y++ < 0 && y < 0) + 2.0`
2. `3.0 + 3 - 4 + 5`
3. `5 % 4 * 3.0 + true * x++`

Gemischte Ausdrücke

Für $x = 1$ und $y = -1$ zu Beginn, was ist der Wert der folgenden Ausdrücke?

1. $(y++ < 0 \ \&\& \ y < 0) + 2.0$
2. $3.0 + 3 - 4 + 5$
3. $5 \% 4 * 3.0 + \text{true} * x++$

Gemischte Ausdrücke (alte Prüfungsfragen)

```
1 int a = 5;  
2 int b = 1;  
3 auto c = (9 * a + b) % a;
```

Geben Sie Typ und Wert der Variable **c** an. / Provide type and value of variable **c**.

Typ / Type:

Wert / Value:

```
1 int a = 5;  
2 double b = 1;  
3 auto c = (9.0 * a + b) / a;
```

Geben Sie Typ und Wert der Variable **c** an. / Provide type and value of variable **c**.

Typ / Type:

Wert / Value:

Tipps zu den Übungen

Equivalent Resistance

- Die Zahlen können in dieser Aufgabe gross sein. Mit einem naiven Ansatz bekommt ihr einen Overflow. Bastelt ein wenig herum, wie genau ihr das $+0.5$ in der Formel implementiert.